Data Copilot PRF

Text-to-SQL com RAG · AWS Bedrock · LangChain · Qdrant · Athena · ECS

Stack AWS

ECS | ECR | EC2 | S3 | Athena | Bedrock | CloudWatch | IAM

Tecnologias

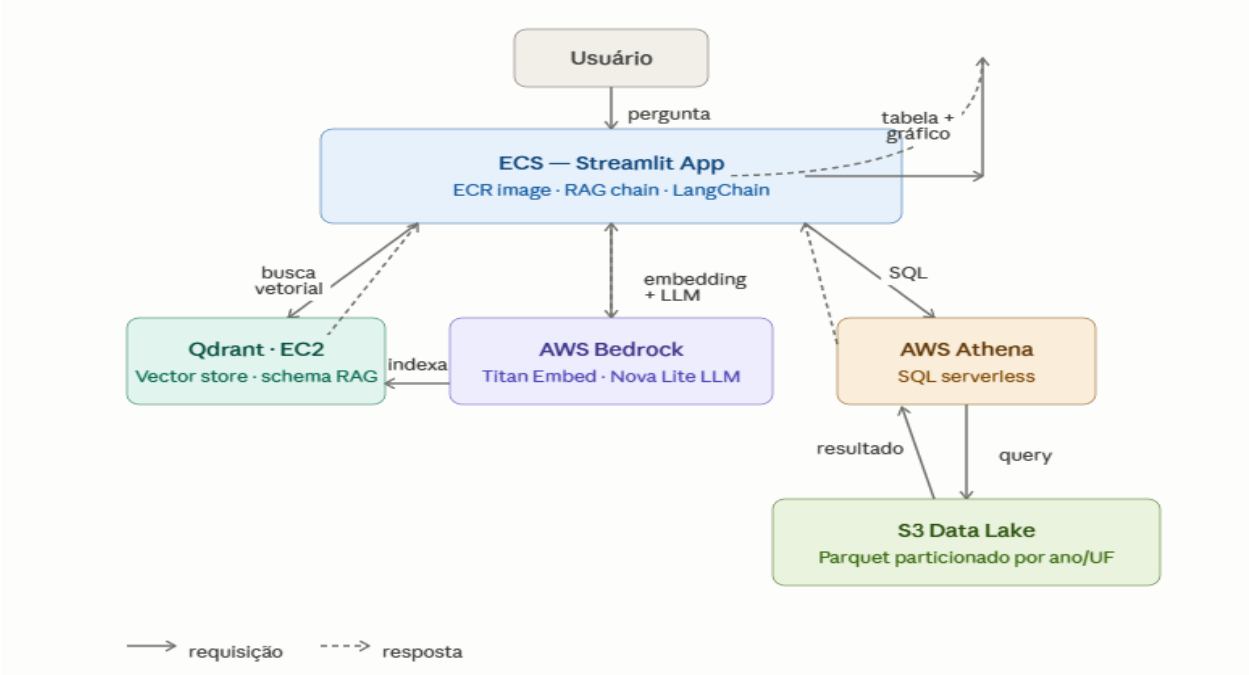
Python | LangChain | Qdrant | Streamlit | Docker | Parquet | RAG | Text-to-SQL

1. O problema que esse projeto resolve

> **Text-to-SQL** permite que qualquer pessoa faça perguntas ao banco de dados em linguagem natural e receba respostas em tempo real — sem conhecimento técnico, sem ticket aberto, com a fonte de dados oficial e o SQL gerado auditável.

A resposta tradicional para democratizar acesso a dados é um dashboard fixo com perguntas pré-definidas. O problema: o negócio muda, as perguntas mudam, e o analista vira gargalo. Com Text-to-SQL o usuário digita — *"top 10 causas de acidentes em SP"* — e recebe a tabela, o gráfico e o SQL gerado na mesma tela.

2. Arquitetura distribuída na AWS

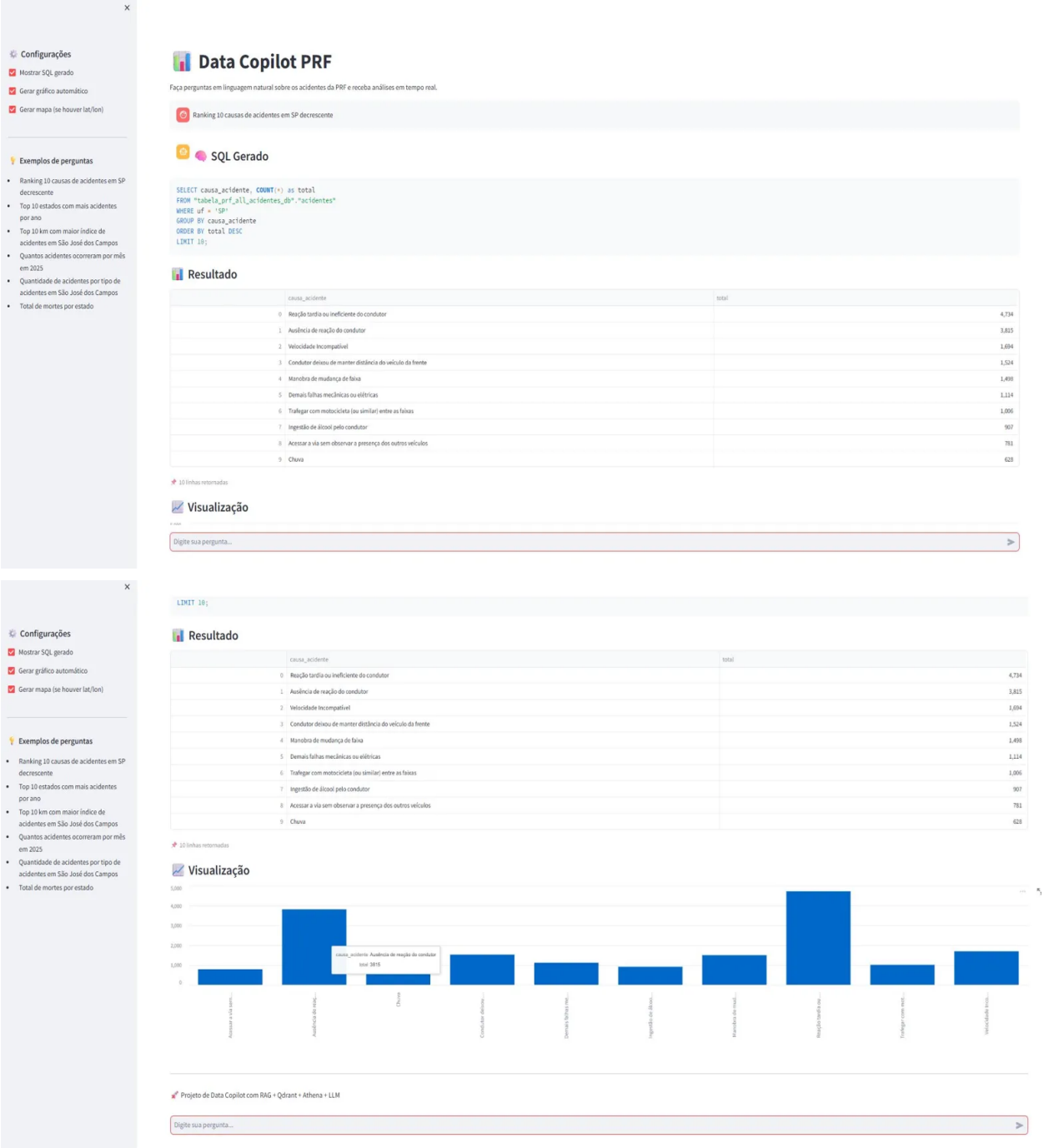


Serviço	Função
ECS + ECR	Runtime da aplicação Streamlit containerizada — stateless, escala sem perda de estado
EC2 + EBS	Banco vetorial Qdrant com volume persistido — stateful, ciclo de vida separado do app
AWS Bedrock	Titan Embed v2 (embeddings 1024d) + Nova Lite (LLM gerador de SQL)
S3 + Athena	Data lake Parquet particionado por ano/UF + SQL serverless pay-per-query

CloudWatch	Logs, CPU e I/O do ECS e EC2 em produção — observabilidade completa
IAM Role	Autenticação entre todos os serviços — zero chaves hardcoded

3. Interface — Streamlit + Docker

Interface construída com **Streamlit** — aplicação de dados interativa em Python sem frontend separado. O usuário faz a pergunta, vê o SQL gerado, a tabela e o gráfico na mesma tela. **Docker** garante paridade entre dev e produção: testado localmente via docker compose, publicado no ECR e deployado no ECS com o mesmo artefato.



4. Decisão arquitetural — EC2 dedicado para o Qdrant

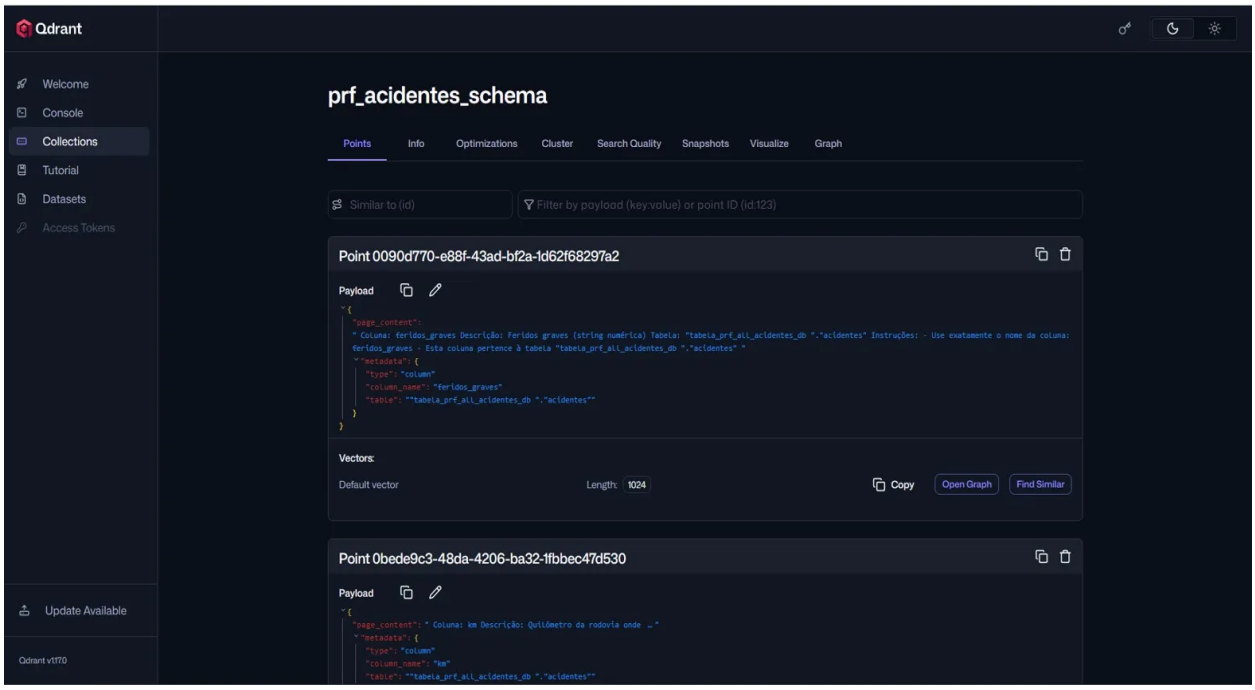
> **Stateless vs Stateful:** ECS recria containers a qualquer momento. Qdrant é stateful — índice vetorial em memória + volume EBS persistido. Juntar os dois causaria perda de dados a cada deploy e impossibilidade de escalar independentemente.

- Deploy do app não derruba o banco vetorial — ciclos de vida independentes
- Collection preservada entre deploys — volume EBS separado
- ECS e EC2 escalam de forma independente conforme a demanda
- Comunicação via IP privado da VPC — sem custo de transferência e sem latência de internet

5. Banco vetorial — Qdrant · Indexação granular por coluna

Cada **coluna da tabela Athena é indexada como um documento vetorial individual**, com nome exato, descrição e instruções de uso. Quando o usuário pergunta sobre feridos graves, a busca recupera especificamente o documento daquela coluna — incluindo a regra de CAST(feridos_graves AS INTEGER). Vetores de **1024 dimensões** · Similaridade coseno · Collection: prf_acidentes_schema.

Tipo de documento	Conteúdo indexado
column (29 docs)	Nome exato, descrição e instrução de uso de cada coluna da tabela
rule (3 docs)	CAST de tipos string, filtros de partição, maiúsculas em municípios/UF
sql_example (4 docs)	Queries completas para casos de uso comuns como ranking e agregações
metric (1 doc)	Padrões de COUNT e SUM para cálculo de totais e agrupamentos



6. HuggingFace → Bedrock: a decisão forçada pela realidade

> **Problema:** O modelo paraphrase-multilingual-MiniLM-L12-v2 ocupa 400–900MB de RAM só para carregar. No ECS causava OOM (Out of Memory) — container derrubado antes de inicializar. Loop de crash: ECS recriava, modelo carregava, OOM, reinício.

> **Solução — Amazon Titan Embed Text v2 via Bedrock:** sem modelo em memória, sem GPU, sem gerenciamento de runtime. Pay-per-token, autenticado via IAM Role. Task ECS leve, custo proporcional ao uso real.

7. Orquestração — LangChain

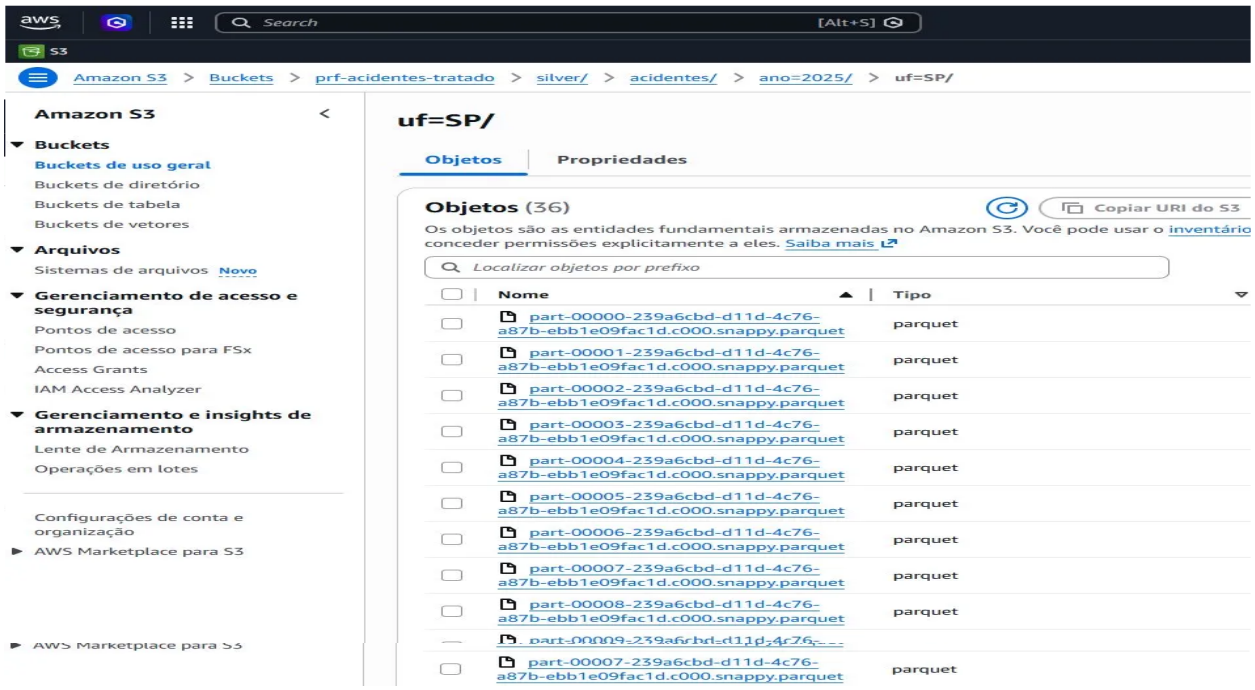
Pipeline declarativo onde cada componente é plugável, testável isoladamente e substituível sem alterar o restante. A chain completa — do input até o SQL pronto — tem menos de 15 linhas.

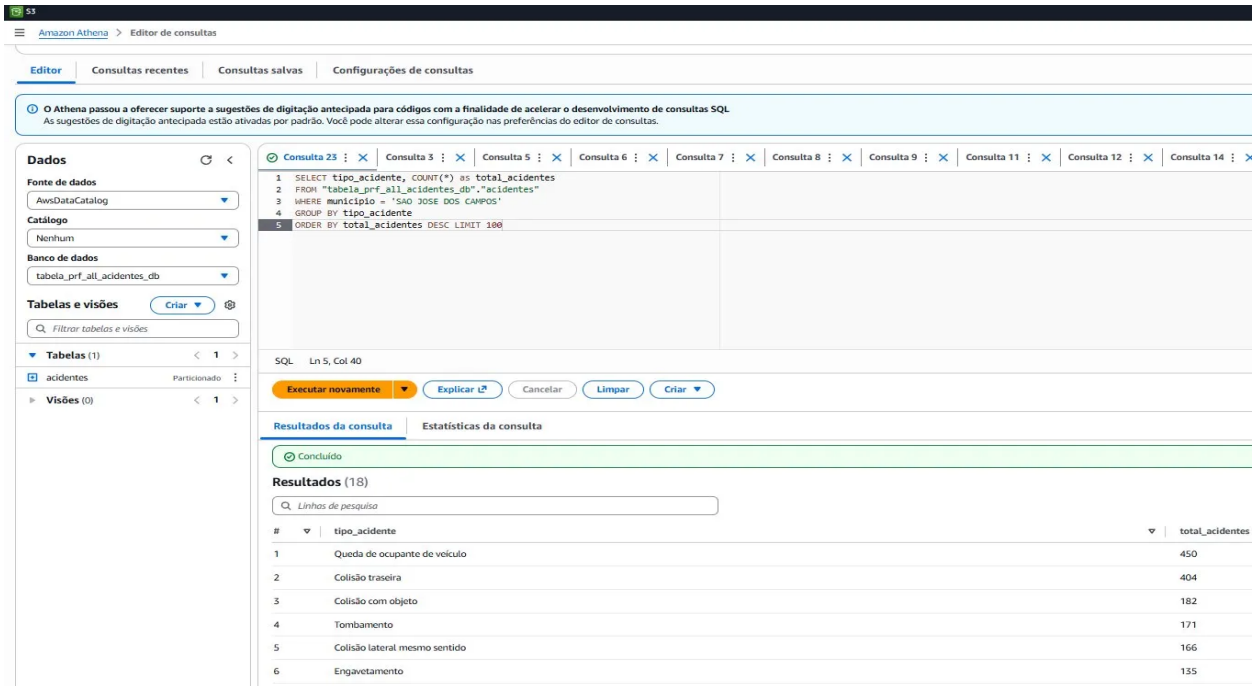
Componente	Função no projeto
RunnableParallel	Busca no Qdrant e preparo da pergunta executados simultaneamente via pipe |
RunnableLambda	Encapsula buscar_contexto como etapa nativa da chain
ChatPromptTemplate	System prompt com regras SQL fixas + human prompt com a pergunta do usuário
RAG filtrado por tipo	10 docs do Qdrant filtrados: 4 colunas + 2 regras + 2 exemplos + 1 métrica
BedrockEmbeddings	Titan Embed v2 como componente LangChain — troca de modelo em uma linha
ChatBedrockConverse	Nova Lite via API Converse — compatível família Amazon Nova + IAM Role
StrOutputParser	Extrai texto e remove markdown antes de executar o SQL no Athena

```
chain = (
    RunnableParallel({
        "pergunta": itemgetter("pergunta"),
        "contexto": itemgetter("pergunta") | RunnableLambda(buscar_contexto),
    })
    | prompt_template
    | model
    | StrOutputParser()
)
```

8. Data lake — S3 + Athena

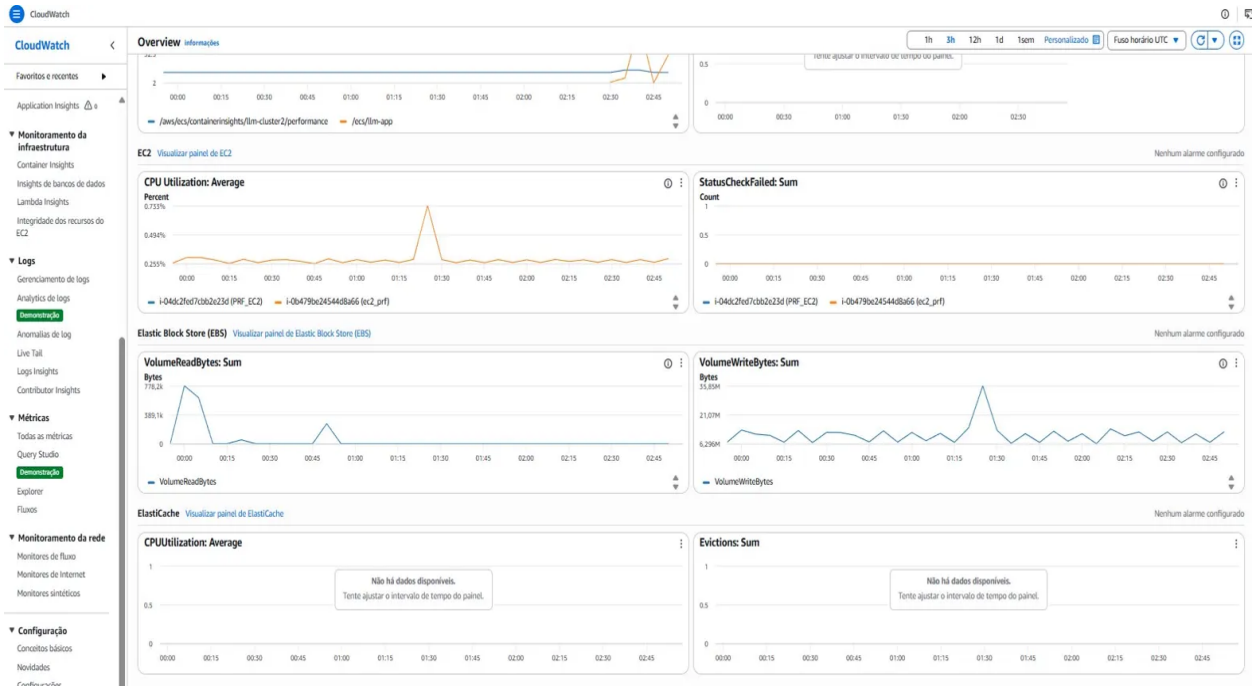
Dados da PRF em **Parquet com estrutura medalhão e particionamento por ano e UF**. Uma query com filtro *ano=2025 AND uf='SP'* lê apenas os arquivos daquela partição — reduzindo volume escaneado, custo e latência. Athena executa as queries geradas pelo LLM de forma serverless.





9. Observabilidade — CloudWatch

Monitoramento em produção: **CPU do ECS e do EC2** (Qdrant), **I/O do EBS** e **logs do container** em tempo real. O pico de CPU no gráfico corresponde ao momento de uma consulta — confirmando que o sistema está respondendo corretamente.



10. O que esse projeto demonstra

Competência demonstrada	Como aparece no projeto
Pipeline RAG para geração de SQL	RAG aplicado a Text-to-SQL — não a Q&A; sobre documentos
Indexação granular no Qdrant	Cada coluna como documento individual com metadados tipados
Decisões arquiteturais reais	HuggingFace OOM → Bedrock · ECS stateless vs EC2 stateful
Orquestração LangChain	RunnableParallel, RunnableLambda, RAG filtrado por tipo de documento
Segurança sem chaves hardcoded	IAM Role na task ECS resolve auth com todos os serviços AWS
Containerização completa	Docker local → docker compose → ECR → ECS — mesmo artefato
Data lake otimizado	Parquet + particionamento por ano/UF para custo e latência controlados
Observabilidade em produção	CloudWatch com CPU, I/O e logs do container em tempo real

---